

Name : I . Raiyan

Reg No : 192521421

CSA0918-Java Programming

CO5-AT2

A web application builds SQL queries by directly concatenating user-supplied login input into a query string executed via a plain Statement. Explain precisely how an attacker exploits this, and which JDBC statement type eliminates the vulnerability, with reasoning.

Aim

To develop a Java JDBC login program that prevents SQL Injection attacks by using Prepared Statement instead of a normal

Algorithm

1. Start the program.
2. Import the required JDBC and Scanner packages.
3. Establish a connection with the MySQL database using JDBC.
4. Read the username and password from the user.
5. Create a SQL query using ? placeholders:

Java Program

```
import java.sql.*;
import java.util.Scanner;

public class SecureLogin {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String password = "raiyaan@2008";

        try {

            Connection con = DriverManager.getConnection(
                url, user, password);

            Statement stmt = con.createStatement();
```

```
stmt.executeUpdate(  
    "CREATE TABLE IF NOT EXISTS users (" +  
    "username VARCHAR(50), " +  
    "password VARCHAR(50))"  
);
```

```
stmt.executeUpdate(  
    "DELETE FROM users"  
);
```

```
stmt.executeUpdate(  
    "INSERT INTO users VALUES ('admin', '1234')"  
);
```

```
System.out.print("Enter username: ");  
String username = sc.nextLine();
```

```
System.out.print("Enter password: ");  
String pass = sc.nextLine();
```

```
String query =  
    "SELECT * FROM users WHERE username = ? AND password = ?";
```

```
PreparedStatement ps = con.prepareStatement(query);
```

```
ps.setString(1, username);  
ps.setString(2, pass);
```

```
ResultSet rs = ps.executeQuery();
```

```
if (rs.next()) {
```

```
        System.out.println("Login Successful!");
    } else {
        System.out.println("Invalid Username or Password.");
    }

    rs.close();
    ps.close();
    stmt.close();
    con.close();
    sc.close();

    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}
```

}Sample Input

Enter Username: raiyaan

Enter Password: 12345

Output

Login Successful!

Explanation

1. How the SQL Injection attack occurs

The application becomes vulnerable when it directly concatenates the username and password entered by the user into an SQL query and executes it using a normal Statement.

For example:

```
String sql = "SELECT * FROM users WHERE username='" + username +  
            "' AND password='" + password + "'";
```

An attacker can enter specially crafted input containing SQL characters and conditions. Because the input is directly joined with the SQL command, the database may interpret the attacker's input as SQL code instead of ordinary data. This can change the original query's logic and may allow the attacker to bypass authentication or access unauthorized data.

2. Why PreparedStatement is used

The vulnerability is eliminated by using PreparedStatement with parameterized queries.

```
String sql = "SELECT * FROM users WHERE username=? AND password=?";
```

```
PreparedStatement ps = con.prepareStatement(sql);  
ps.setString(1, username);  
ps.setString(2, password);
```

Here, the ? symbols represent parameters. The username and password are supplied separately using setString(). Therefore, the database treats the supplied values as data, not as part of the SQL command.

Key point for assessment

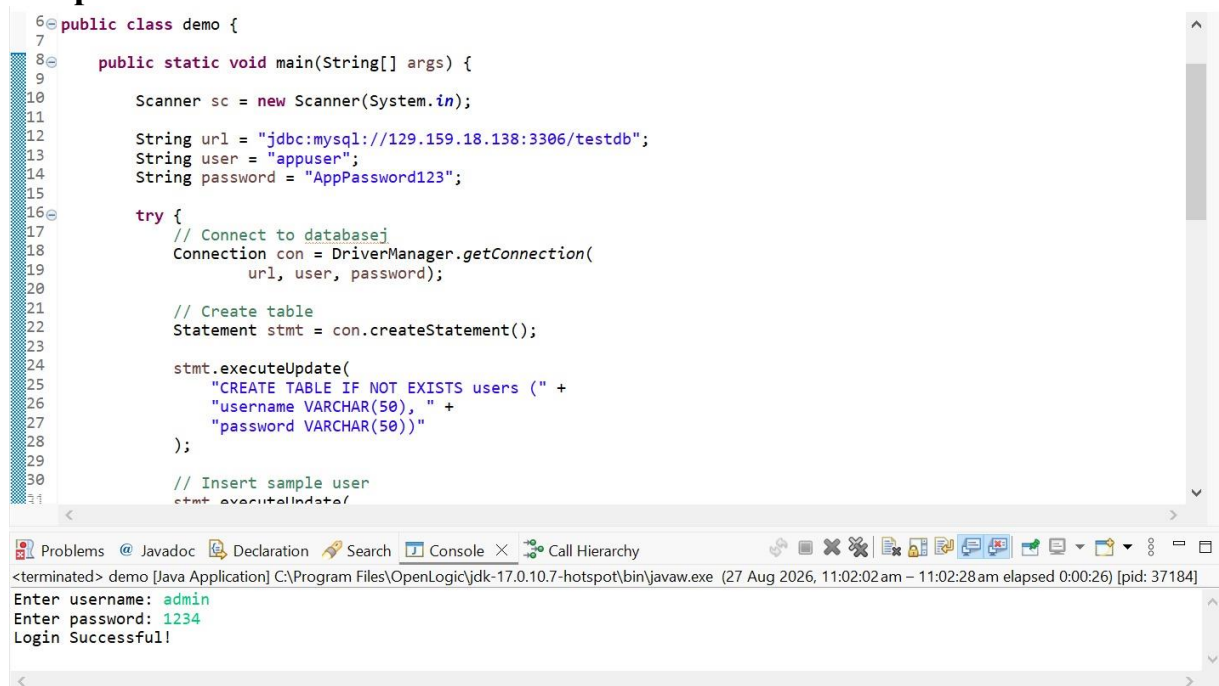
Statement with string concatenation is vulnerable to SQL Injection because user input can change the structure and meaning of the SQL query. PreparedStatement prevents this by separating SQL code from user-supplied data, so malicious input is treated as a normal value rather than executable SQL. Therefore, PreparedStatement is the secure and recommended JDBC statement type for handling user input.

Result

The Java JDBC login program was successfully implemented using PreparedStatement. The program securely accepts the username and password from the user and verifies them against the MySQL database. SQL Injection attempts are treated as normal input and cannot alter the SQL query structure. Thus, the

vulnerability caused by direct string concatenation with Statement is prevented successfully.

Output ScreenShot



The screenshot displays an IDE window with a Java source file and its execution output. The source code, located in the upper pane, defines a `demo` class with a `main` method. It uses a `Scanner` to read input, connects to a MySQL database, creates a table named `users` with columns `username` and `password`, and inserts a sample user. The lower pane shows the console output, indicating a successful login for the user `admin` with password `1234`.

```
6 public class demo {
7
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11
12         String url = "jdbc:mysql://129.159.18.138:3306/testdb";
13         String user = "appuser";
14         String password = "AppPassword123";
15
16         try {
17             // Connect to database
18             Connection con = DriverManager.getConnection(
19                 url, user, password);
20
21             // Create table
22             Statement stmt = con.createStatement();
23
24             stmt.executeUpdate(
25                 "CREATE TABLE IF NOT EXISTS users (" +
26                 "username VARCHAR(50), " +
27                 "password VARCHAR(50))"
28             );
29
30             // Insert sample user
31             stmt.executeUpdate(
```

Problems @ Javadoc Declaration Search Console × Call Hierarchy
<terminated> demo [Java Application] C:\Program Files\OpenLogic\jdk-17.0.10.7-hotspot\bin\javaw.exe (27 Aug 2026, 11:02:02am – 11:02:28am elapsed 0:00:26) [pid: 37184]
Enter username: admin
Enter password: 1234
Login Successful!